

SAIR Stage1 Cheatsheet 表现复盘简报

日期：2026-05-12

1. 背景

本报告是对我在 SAIR Stage1 Equational Theories 任务中最终提交结果的一次简洁复盘。任务要求模型判断两个代数等式之间是否存在“必然推出”关系：如果所有满足 Equation 1 的代数结构也满足 Equation 2，则输出 `true`；如果存在一个反例结构，则输出 `false`。

我的公开榜单记录为：

项目	数值
Team	AscendFoam
Rank	274
Cheatsheet Size	3.91 KB
Avg Accuracy	51.6%
Avg F1	31.3%
Parse Rate	100.0%
Avg Cost	\$0.00009

这个结果有一个明显矛盾：格式解析完全稳定，平均成本非常低，但准确率和 F1 都偏低。本文主要解释这个现象，并比较第一名方案为什么明显更好。

2. 结论摘要

第一，我的 cheatsheet 不是因为“太长”而失败，而是因为它没有把规则写成足够稳定、可执行的判别流程。它更像是一组自然语言提示，让模型快速判断；第一名的 cheatsheet 更像一个小型手工分类器，要求模型先解析等式结构，再按顺序应用规则。

第二，我的低成本主要来自输出很短。我要求模型最终只输出 `true` 或 `false`，没有要求它写出解析过程、规则轨迹或反例说明。因此模型可以很快结束推理，成本自然很低。但这也意味着它没有被强制

做充分的结构检查。

第三，几乎空白的 cheatsheet 有时反而表现不错，是因为强模型本身已经有一定判断能力。如果额外提示没有提供稳定的结构规则，反而可能干扰模型原有判断。这也解释了为什么我的 3.91 KB prompt 并没有明显超过一些接近空白的 prompt。

第四，9 KB 左右的 cheatsheet 也不一定好。长度本身不是关键，关键是这些文字是否真的变成了可执行的判断步骤。第一名的长 prompt 有大量结构特征、规则编号、反例检测和回退决策树；一些较长但效果差的 prompt 可能只是堆叠说明或经验规则。

第五，一个重要的实验设计问题是：我前期几乎都在 DeepSeek 上迭代，直到末期才去官方 playground 上测试 GPT-OSS、Gemma、Llama 等模型。这导致 prompt 在 DeepSeek 上看起来有改进，但没有足够早地暴露跨模型不稳定问题。

3. 我的方案表现为什么低

我的最终提交大致对应仓库中的 P1.2.5_minimal_rule_missing_hard_composition 。这个版本在本地 DeepSeek 实验中并不是最稳健的主线，而是一个“提高 true 召回”的分支。

本地 smoke 集上，原保守版本 P1_2_3 的特点是：

Prompt	Accuracy	True Accuracy	False Accuracy
P1_2_3	54.69%	22.58%	84.85%

它的问题是过于保守，很多本该判 true 的题被判成 false 。

后来我尝试的 P1_2_5 能补回更多 true：

Prompt	Accuracy	True Accuracy	False Accuracy
P1_2_5	57.81%	64.52%	51.52%

这说明 P1_2_5 的确提高了 true 侧表现，但代价是 false 侧下降明显。换句话说，它不是一个完全平衡的方案，而是一个为了修补 true 漏判而产生的高召回分支。

最终选择 P1_2_5 ，主要是因为末期官方 playground 小样本显示， P1_2_3 在官方模型组合上 true 漏判更严重，而 P1_2_5 看起来更适合提交窗口。但这个判断建立在很小的官方样本上，风险较大。

4. 低成本为什么反而可能是问题信号

公开榜单中，我的 avg cost 是 $\$0.00009$ ，几乎是全榜最低档；而第一名的平均成本大约在 $\$0.0006$ 左右，截图中各模型成本也通常比我高 6 到 8 倍。

成本低有两个原因。

一是输出格式极短。我的 prompt 要求最终只输出一个 token: `true` 或 `false`。第一名方案要求模型先写 parse 结果，再写规则名、证明或反例说明，输出自然更长。

二是我的 prompt 允许模型很快作出判断。例如它告诉模型：先检查一些 mandatory true rules；如果命中就立刻输出 true；如果看不到短推导就倾向 false。这种设计非常省 token，但也容易让模型在没有充分验证的情况下提前结束。

因此，我的低成本并不说明“同样质量下更高效”，更可能说明模型没有展开足够稳定的结构分析。

5. 为什么几乎空白 prompt 也可能比我好

从榜单统计看， ≤ 0.2 KB 的极短 cheatsheet 平均 accuracy 约为 51.68%，平均 F1 约为 37.8%。我的结果是 51.6% accuracy、31.3% F1。也就是说，我的 3.91 KB prompt 在 accuracy 上几乎没有超过极短组，在 F1 上还更低。

这说明我加入的规则没有稳定转化为有效判断能力。

这类任务与普通问答不同。模型不只是需要理解文字，还要判断抽象代数关系。提示词如果只是告诉模型“遇到某些形式要更倾向 true/false”，但没有强制它先做结构化检查，就可能变成一种干扰。

Less is more 论文中也提到类似现象：prompt 不是越复杂越好。过多规则可能造成认知负担，尤其在不同模型之间迁移时更容易失效。一个短 prompt 有时能保留模型自身的判断方式，反而不容易被错误规则带偏。

6. 为什么 9 KB prompt 也可能比我差

榜单中确实有一些 9 KB 左右的 prompt 表现很好，例如第一名；但也有一些 9 KB 左右的 prompt 表现很差。

原因是长度本身并不代表质量。一个长 prompt 可能有两种情况：

类型	特点	可能结果
有效长 prompt	把规则写成清晰步骤、结构特征、反例检测、回退判断	可能显著提升
无效长 prompt	堆叠经验、解释和互相冲突的规则	可能干扰模型

所以，9 KB 不是成功原因。真正重要的是这些字节是否变成了“模型可以稳定执行的判断流程”。

7. 第一名方案和我的区别

第一名的 cheatsheet 大约 9.83 KB，公开榜单为：

Team	Rank	Accuracy	F1	Parse Rate	Cost
Dufius	1	69.1%	65.0%	99.5%	\$0.00060
AscendFoam	274	51.6%	31.3%	100.0%	\$0.00009

第一名在 GPT-OSS 和 Gemma 上表现很强，在 Llama 上虽然下降，但没有像我的方案那样 F1 接近塌缩。

第一名方案最重要的特点有四个。

第一，它强制模型先做 parse。它要求模型对 Equation 1 和 Equation 2 的四个侧分别写出变量集合、变量出现次数、乘法节点数量、左右端变量、左右深度等信息。这等于要求模型先把公式变成结构化对象。

第二，它把判断规则编号化。例如 X1、F1、C1、S1、A1、B1 等，每个规则都有明确触发条件，并要求“按顺序执行，命中第一个就停止”。这比自然语言提示更像一个确定流程。

第三，它有较强的 false 检测机制。除了 true collapse 类规则外，它还设计了 separator rules 和 affine probes。简单说，它不仅问“这个结论看起来是否能推出”，还尝试构造一些小型反例模式。如果 Equation 1 在某个小结构下成立而 Equation 2 不成立，就可以判 false。

第四，它有 fallback decision tree。当前面规则都不命中时，它不是简单让模型凭感觉判断，而是继续根据 Equation 1 的变量重复情况、左侧复杂度等特征做最终分类。

我的 P1_2_5 相比之下更像自然语言版的判断建议。它告诉模型先检查 collapse、substitution、same-left-hand-side 等情况，但没有强制模型把变量计数、左右端点、深度等中间特征显式算出来。

因此，第一名方案的优势不是“文字更多”，而是“结构更像程序”。

8. DeepSeek 单模型迭代带来的风险

我这次实验有一个重要局限：主要迭代过程都在 DeepSeek 上完成。

这样做有现实原因：DeepSeek 成本较低，调用方便，适合快速做 prompt 实验。但副作用是，我的 prompt 逐渐适应了 DeepSeek 的行为习惯，而不一定适合官方评测中的多模型组合。

从项目记录看，直到最后阶段，我才去官方 playground 上测试 GPT-OSS、Gemma、Llama 等模型。这个时间点太晚，已经来不及重新设计一个真正跨模型稳健的 prompt。

这可能是最终表现不好的重要原因之一：

- DeepSeek 上有效的 wording，不一定能迁移到 GPT-OSS。
- DeepSeek 上能保持的 true/false 平衡，在 Llama 上可能直接变成单标签倾向。
- DeepSeek 上看起来合理的“短推导可见”判断，在其他模型上可能理解完全不同。

因此，这次失败不只是 prompt 内容问题，也是实验验证路径问题。前期如果能更早引入官方模型或近似官方模型，可能会更早发现 P1_2_5 的跨模型不稳定。

9. 对第一名的简要分析

第一名截图中，GPT-OSS 和 Gemma 的 normal/hard 成绩都在 80% 以上，F1 也接近 accuracy。这说明它不是单纯偏向 true 或 false，而是两类标签都处理得比较平衡。

我的截图中，Llama 的 F1 在 normal/hard/extra_hard 上分别只有 1.3%、6.4%、0.0%，这非常像单标签输出或严重标签失衡。GPT-OSS 在 normal/hard 上 F1 也只有 18.1%、18.9%，说明它虽然 parse 成功，但判断几乎没有形成健康的二分类能力。

一个特别值得注意的点是：我在 extra_hard 上 GPT-OSS accuracy 达到 63.7%，略高于第一名的 62.5%，但 F1 只有 50.7%，明显低于第一名的 66.6%。这说明我可能在这个 split 上碰巧押中了更多样本，但整体标签平衡仍然不好。

所以，单看某个 accuracy 可能会误判。F1 更能暴露模型是否真的同时处理好了 true 和 false。

10. 理论改进方向

虽然Stage1比赛已经结束，但是可行的改进方向是，不应该简单继续加自然语言规则，也不应该直接照抄第一名的长 prompt。而是吸收第一名的结构思想，做一个更轻量的版本。

比如：

1. 保留低成本优势，但加入少量 parse-first 步骤。
2. 把“看起来更强”“没有短推导”这类模糊表达，改成更明确的结构特征。
3. 加入少量可靠的 false separator，用于避免 true 侧过度放行。
4. 给 fallback 一个小决策树，而不是只说“看不到推导就 false”。
5. 从实验一开始就至少用两个模型交叉验证，避免只适配 DeepSeek。
6. 把 prompt 分成“规则发现用”和“正式提交用”两类，不要把高召回探索 prompt 直接当最终提交 prompt。

11. 总结

我的最终提交失败的主要原因，不是 prompt 太短或太长，而是规则没有被组织成足够稳定的结构化判别流程。它在 DeepSeek 本地实验中能补回一部分 true，但牺牲了 false 侧稳定性；在官方多模型环境中，这种不平衡被进一步放大。

第一名方案的优势也不只是信息更多，而是把任务变成了更明确的“先解析、再按规则分类”。它用更高成本换来了更稳定的中间过程和更健康的 F1。

这次结果给我的主要经验是：对于这类形式推理任务，prompt 不是越长越好，也不是越便宜越好。关键是要让模型执行一个清楚、稳定、可迁移的判断流程，并且在实验早期就用接近官方设置的多模型环境验证。